OFFLINE AIRPLANE READING EDITION · HIGH-LEVEL + TECHNICAL CHAPTERS

How Coding Models Are Built & Trained (Mobile App Development Lens)

Bridging from Financial Sentiment Analysis on Historical Data to Transformer Code Generation, Mathematical Objectives, Code Tokenization (BPE & FIM), Execution Evals (pass@k & SWE-bench), and Custom Fine-Tuning vs. AGENTS.md & Skills.

PART I — HIGH-LEVEL INTUITION & ARCHITECTURE

Chapter 1: From Stock Sentiment to Coding Models (The Mental Bridge)

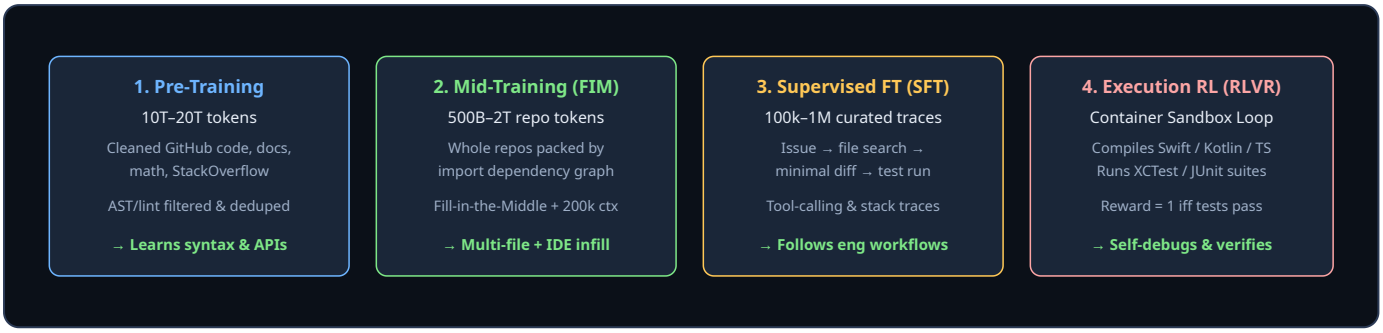
Since you have experience building a **sentiment analysis model for publicly traded companies on historical data** (such as 10-K/10-Q filings, earnings call transcripts, and financial news), you already know the core foundations. Moving from stock sentiment to a mobile coding LLM changes three things: **what a single prediction step outputs**, **how tokens are masked**, and **how ground-truth rewards are verified**.



Concept in Your Stock Sentiment Model	Exact Parallel in a Mobile Coding Model
3-Class Output (`Bearish` , `Neutral` , `Bullish`)	**~100,000-Class Output** (every subword/code token in the vocabulary) predicted one token at a time in a loop.
Historical Corpus (10-K filings, earnings transcripts, news)	**Code + Engineering Corpus** (Swift, Kotlin, TypeScript repos, PR diffs, commit messages, compiler error logs).
Lookahead Bias (training on 2024 data and backtesting on 2023)	**Benchmark Contamination & Temporal Leakage** (leaking test suite solutions or mixing incompatible SDK versions).
Market Regime Shift (zero-rate era model failing in high-rate regime)	**Mobile SDK Deprecation Drift** (model trained on older `UIKit` / `NavigationView` or Android XML instead of `SwiftUI @Observable` / `Jetpack Compose`).
Noisy Market Return Signal (good earnings still drop 4% on Fed news)	**Deterministic Compiler & Unit Test Signal** (`xcodebuild test` / `./gradlew test` gives a 100% noise-free verifiable reward!).

Chapter 2: How a Coding Model is Built (The 4-Stage Factory Pipeline)

A coding model is built through four sequential training phases. Each stage transforms the weights from raw pattern recognition into an autonomous software engineering assistant:



- **Stage 1 (Pre-Training):** Ingests trillions of tokens of public code after aggressive filtering (removing auto-generated `.pbxproj` blobs, minified JS, and broken forks). Teaches language grammar, standard libraries, and general reasoning.
- **Stage 2 (Repo-Level Mid-Training & FIM):** Concatenates related files from the same repository (e.g., `User.swift` → `UserRepository.swift` → `ProfileViewModel.swift` → `ProfileView.swift`) and extends the context window from 4k/8k out to 128k–200k+ tokens. Also trains with the **Fill-in-the-Middle (FIM)** objective so the model can insert code in the middle of an existing file.
- **Stage 3 (Supervised Fine-Tuning / SFT):** Trains on multi-step agent trajectories where an engineer or teacher model uses search tools, reads files, edits code via diffs, and interprets compiler errors.
- **Stage 4 (Reinforcement Learning with Verifiable Rewards / RLVR):** The model attempts thousands of real coding bugs inside isolated build containers. When its generated code compiles and passes unit tests, those reasoning chains and token choices are reinforced.

Chapter 3: What Makes Mobile App Development Unique for Coding Models

1. Rapid SDK Deprecation ("Regime Shift")

iOS and Android APIs deprecate yearly. Older training data is heavy on `UIViewController`, `NavigationView`, and Android XML/`AsyncTask`, whereas modern codebases require `SwiftUI` `@Observable`, `NavigationStack`, Swift 6 strict concurrency, and `Jetpack Compose`.

2. Cross-Layer Configuration Coupling

Adding a single mobile feature (like deep linking or camera access) spans UI code, `Info.plist` / `AndroidManifest.xml` permissions, Gradle/SPM dependency files, and native bridge modules.

3. Lifecycle, Main-Thread & 120Hz UI

Mobile apps are stateful and long-lived. Models must avoid retain cycles (`[weak self]`), unnecessary Compose recompositions, and blocking the main thread (`@MainActor` / `Dispatchers.IO`).

4. Multimodal 2D Layout Reasoning

Code is 1D text, but mobile UI renders on varied screens with notches, Safe Areas, Dynamic Type font scaling, and soft keyboards. Frontier models pair screenshots/Figma trees with SwiftUI/Compose code during training.

Chapter 4: Existing Codebases — Custom Model Training vs. `AGENTS.md` & Skills

Your Question: *"Can a new model be created or tweaked or trained specifically for an existing codebase? Or is it more of an `AGENTS.md` and skills department?"*

> **Core Engineering Verdict:** For **95% of existing mobile codebases**, adapting AI to your repository is **the department of `AGENTS.md`, Skills (`SKILL.md`), and Codebase Indexing (RAG/Search)** — *not* training or fine-tuning a custom model.

Why Fine-Tuning Weights on a Living Codebase Usually Backfires

Think back to your historical stock sentiment model: if you baked 2021 company facts into the neural network weights, those weights became stale the moment companies merged or changed business models in 2023. In software engineering, **your codebase changes every day via `git commit`**:

- **Weight Staleness:** If you fine-tune a model on Monday and your team refactors `AuthManager` into `SessionCoordinator` on Wednesday, the fine-tuned weights will keep hallucinating calls to the deleted `AuthManager`.
- **Fuzzy Parametric Memory vs. Exact Context:** Neural weights are statistical compressors — great for learning *how to write idiomatic Swift or Kotlin*, terrible for memorizing exact file paths and today's function signatures.

The 4-Layer Architecture for an Existing Mobile Codebase

Layer	Mechanism	What Belongs Here (Mobile Example)	When to Use
1. Base Frontier Model	Pre-trained + RLVR weights (frozen at inference)	General Swift, Kotlin, React Native, architecture, debugging & reasoning skills.	Always (your foundation engine).
2. `AGENTS.md` & Rules	Always-on system prompt conditioning (checked into git)	Repo constitution: *"Target iOS 17+, use @Observable, MVVM-C pattern, run `./gradlew testDebugUnitTest`, never use NavigationView."*	**#1 Highest ROI** for every existing codebase.
3. Skills (`SKILL.md`)	On-demand procedural playbooks loaded only when triggered	Multi-step SOPs: *"How to scaffold a new Feature Module"*, *"How to add localized strings & run SwiftGen"*, *"How to update snapshot tests."*	**#1 Highest ROI** for repeatable team workflows without bloating token context on unrelated turns.
4. Codebase Index + Tools	Vector embeddings + AST chunking + `grep` / LSP	Retrieving the exact live definition of `CheckoutViewModel.kt` and its callers on the current git branch.	Essential on every multi-file codebase.
5. Custom Fine-Tuning (LoRA)	Low-rank weight adapters ($W + B \cdot A$)	Proprietary non-public languages (internal DSLs), tiny sub-100ms Tab-autocomplete models, or air-gapped local SLMs.	Only ~5% of teams (and still paired with `AGENTS.md` + RAG).

PART II — TECHNICAL DEEP-DIVE, MATH & ALGORITHMS

Chapter 5: The Math — Sentiment Classification vs. Autoregressive Code Generation

1. Your Financial Sentiment Model (Bidirectional Encoder + 3-Class Softmax)

```
Input sequence:  X = (x_1, x_2, ..., x_T)
Embeddings:      h_t^(0) = E[x_t] + p_t,    where E ∈ ℝ^(|V| × d)
Self-Attention:  Attn(Q, K, V) = softmax( (Q · K^T) / √d_k ) · V   (Bidirectional: every token sees all tokens)
Pooled Vector:   h_CLS ∈ ℝ^d  → projected via W_cls ∈ ℝ^(3 × d) to C = 3 classes (Bearish, Neutral, Bullish)
Class Prob:      P(y = c | X) = exp(w_c^T · h_CLS + b_c) / Σ_{j=1}^3 exp(w_j^T · h_CLS + b_j)
Training Loss:   L_sentiment(θ) = - (1/N) Σ_{i=1}^N Σ_{c=1}^3 y_{i,c} log P_θ(y = c | X_i)
```

2. Coding LLM Pre-Training (Causal Decoder + 100,000-Way Next-Token Prediction)

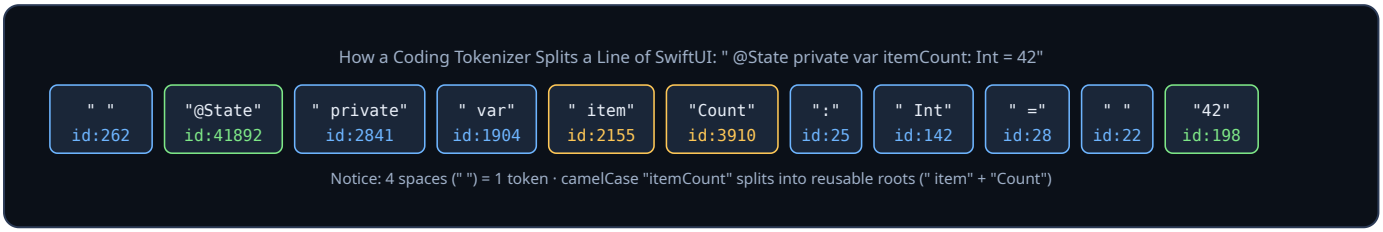
```
Causal Mask M:   M_{i,j} = 0 if j ≤ i, else -∞   (Token t can ONLY attend to past tokens ≤ t)
Causal Attn:     CausalAttn(Q, K, V) = softmax( (Q · K^T) / √d_k + M ) · V
RoPE Position:   Rotates Query/Key vectors by angle m·θ_k so dot product ⟨R_m q, R_n k⟩ depends only on relative distance (m - n)
Next-Token Prob: P_θ(x_t | x_{<t}) = softmax( W_u · h_{t-1}^(L) )_{x_t},    where |V| ≈ 100,000 to 200,000 tokens
Sequence Prob:   P_θ(x_1, ..., x_T) = Π_{t=1}^T P_θ(x_t | x_1, ..., x_{t-1})
Training Loss:   L_CLM(θ) = - Σ_{t=1}^T log P_θ(x_t | x_1, x_2, ..., x_{t-1})
```

Why this mathematical difference matters: In your sentiment model, one earnings call transcript gives you **1 supervision signal** (1 label of ~1.58 bits of entropy). In a coding LLM, a single 1,000-token Swift file gives the model **1,000 dense classification targets in parallel** during a single forward/backward pass!

Chapter 6: Tokens in Code & Fill-in-the-Middle (FIM) Mechanics

1. Byte-Pair Encoding (BPE) for Source Code

Models never see raw characters; they see integer token IDs produced by a **Byte-Pair Encoding (BPE)** tokenizer. Starting from 256 UTF-8 byte tokens, BPE greedily merges the most frequent byte pairs in a code corpus until reaching vocabulary size `|V| ≈ 100,000+`.



2. Fill-in-the-Middle (FIM) Objective

How does a strictly left-to-right causal decoder autocomplete code in the middle of a SwiftUI `View` when it also needs to see the closing braces and state properties *below* the cursor? During training, ~50% of files are split into `(Prefix, Middle, Suffix)` and rearranged with sentinel tokens:

Fill-in-the-Middle (FIM) Sequence Transformation

```
Original File:   [Prefix P (above cursor)] + [Middle M (missing span)] + [Suffix S (below cursor)]
Transformed:     ◁fim_prefix▷ P ◁fim_suffix▷ S ◁fim_middle▷ M ◁endoftext▷
FIM Objective:   L_FIM(θ) = - Σ_{t=1}^{|M|} log P_θ(m_t | P, S, m_1, ..., m_{t-1})
```

Chapter 7: Post-Training Math — SFT, DPO, and Execution RL (GRPO)

After pre-training, the model knows how to complete code, but not how to verify correctness. Modern coding models apply **Group Relative Policy Optimization (GRPO)** with verifiable execution rewards:

Group Relative Policy Optimization (GRPO) with Compiler & Unit Test Rewards

```
1. Sample G candidate patches {o_1, ..., o_G} ~ π_{θ_old}(· | q) for coding issue q.
2. Execute each patch in an isolated container (e.g., xcodebuild test / ./gradlew test)
to get verifiable reward:
      r_i = 0.2 · I[Compiles] + 0.8 · (Tests_Passed / Total_Tests) - c_lint
3. Compute group-normalized advantage (no separate critic network required!):
      A_i = (r_i - mean(r_1, ..., r_G)) / (std(r_1, ..., r_G) + ε)
4. Policy update with KL regularization toward reference model π_ref:
      J_GRPO(θ) = (1/G) Σ_{i=1}^G [ min( ratio_i · A_i, clip(ratio_i, 1-ε, 1+ε) · A_i ) -
β · D_KL(π_θ || π_ref) ]
```

Chapter 8: What Are "Evals" in Coding & Mobile Development?

In financial sentiment modeling, you evaluate with static classification metrics (Precision, Recall, F1, ROC-AUC) and backtest Sharpe ratio. In coding, static string similarity (BLEU/ROUGE) fails because 10 syntactically different Swift functions can implement the exact same behavior. Instead, coding uses **Execution Evals**:

The Unbiased pass@k Estimator (Chen et al.)

```
Generate n ≥ k candidate solutions per problem; let c be the number of candidates that compile and pass 100% of unit tests:
      pass@k = 1 - C(n - c, k) / C(n, k)
Interpretation: The exact probability that at least 1 of k sampled solutions passes every test in the test suite.
```

- **Function-Level Evals (HumanEval, MBPP, MultiPL-E):** Single function signature + docstring → run hidden unit tests in Swift, Kotlin, TypeScript, Python.
- **Repository-Level Evals (SWE-bench Verified):** Real GitHub repository + real issue. Graded on two test sets: `Fail-to-Pass` (reproduces the issue before the patch, passes after) and `Pass-to-Pass` (zero regressions on existing tests).
- **Mobile-Specific Eval Gates:** (1) Static/Lint check (SwiftLint/Detekt), (2) Build check (`xcodebuild` / `gradlew assembleDebug`), (3) Unit/ViewModel test check (`XCTest` / `JUnit`), (4) Snapshot / UI test check (Paparazzi / `swift-snapshot-testing` / `XCUITest`).

Chapter 9: The Math of Adapting to Your Codebase — LoRA vs. In-Context Conditioning

Parametric Adaptation (LoRA) vs. Non-Parametric Conditioning (AGENTS.md + Skills + RAG)

```
• LoRA Fine-Tuning (Modifies Weights):
      h = W_0 x + ΔW x = W_0 x + (α / r) · B · A · x,     where B ∈ ℝ^(d×r), A ∈ ℝ^(r×k), r ≪ d
  Pros: Teaches new syntax/DSLs.  Cons: Goes stale on every git commit; expensive to maintain.

• In-Context Conditioning (Modifies Attention KV Cache at Inference):
      P_θ( Y  |  C_{AGENTS.md},  C_{SKILL.md},  C_{RAG/Grep},  X_{prompt} )
  Tokens in AGENTS.md, Skills, and retrieved files become Key/Value vectors (K_ctx, V_ctx) in every attention layer.
  Every generated token attends directly to K_ctx and pulls your exact repo conventions & live types from V_ctx!
```